

Configuration Reference

Environment variables and runtime settings

Document	6 of 9
Audience	Developers & DevOps
Revision	Rev 1.0 · 2026
Status	Handoff

Configuration Reference — Castillo Planset QC

This document is the definitive environment-variable reference for the Castillo Planset QC application. It is intended for the Developer running the FastAPI backend locally and the DevOps/Automation Engineer operating the Azure Container Apps deployments (prod: `castillo-qaqc-automation`; sandbox: `castillo-qaqc-sandbox`).

All values below are read directly from the application code. Variable names, defaults, and fallback behavior were confirmed against `backend/app/gemini_client.py`, `backend/app/jobs.py`, `backend/app/auth.py`, `backend/app/rule_registry.py`, `backend/app/db.py`, `backend/app/main.py`, and `backend/.env.example`.

Secrets note: Never commit real keys. `backend/.env` is gitignored. In Azure, credential-bearing variables (API keys) are stored as Container App **secrets** and referenced by env vars; non-secret tuning values are stored as plain env vars. This document never reproduces secret values.

How configuration is loaded

- **Backend (local):** `backend/app/gemini_client.py` calls `load_dotenv(backend/.env)` at import time, so a `backend/.env` file populates `os.environ` for the whole process. Other modules simply read `os.getenv(...)`. Copy `backend/.env.example` → `backend/.env` to start.
- **Backend (Azure):** No `.env` file ships in the image. Values are injected as Container App environment variables and secrets, read through the same `os.getenv(...)` calls.
- **Precedence:** A real OS/Container environment variable always wins; `.env` only fills in what the environment does not already set.
- **Most AI knobs are read at call time**, not just at import (e.g. `OPENAI_SEED` in `_openai_sampling_kwargs()`), so they can be toggled per process restart without code changes.

Complete environment-variable reference

Variable	Purpose	Default	Where set
<code>AI_PROVIDER</code>	Selects the vision/LLM backend. Lowercased; <code>openai</code> uses the OpenAI client, anything else (<code>gemini</code>) uses the Gemini client.	<code>openai</code>	Local <code>.env</code> and Azure env (plain)

OPENAI_API_KEY	OpenAI API credential. Required when AI_PROVIDER=openai ; a missing key raises RuntimeError on first call.	"" (empty)	Local .env ; Azure secret
OPENAI_MODEL	Model ID for standard (per- page) OpenAI checks.	gpt-4.1-mini	Local .env ; Azure env (plain)
OPENAI_MODEL_DEEP	Model ID for deep/reasoning passes (SLD validation, cross-sheet consistency). Empty value falls back to OPENAI_MODEL .	"" → falls back to OPENAI_MODEL	Local .env ; Azure env (plain)
OPENAI_SEED	Optional integer seed passed to OpenAI chat- completions for best-effort determinism (reproducible runs, clean before/after diffs). Non-integer or empty → normal sampling. Not a hard guarantee — an infra/model change (system_fingerprint) can still shift output, and it does not resolve genuine model judgment on ambiguous drawings.	unset → normal sampling	Local .env ; Azure env (plain)
GEMINI_API_KEY	Gemini API credential. Required when AI_PROVIDER=gemini ; a missing key raises RuntimeError on first call.	"" (empty)	Local .env ; Azure secret (only if Gemini is used)
GEMINI_MODEL	Model ID for standard (per- page) Gemini checks.	gemini-2.5-flash	Local .env ; Azure env (plain)
GEMINI_MODEL_DEEP	Model ID for deep Gemini passes. Empty value falls back to GEMINI_MODEL .	"" → falls back to GEMINI_MODEL	Local .env ; Azure env (plain)
ANALYZE_CONCURRENCY	Max analyses (jobs) running concurrently in the bounded ThreadPoolExecutor .	2	Local .env ; Azure env (plain)

	Clamped to a minimum of 1 via <code>max(1, int(...))</code> . Extra submissions wait as <code>queued</code> .		
AI_PARALLELISM	Worker count for the per-page/diagram fan-out inside a single analysis (thread pool in <code>gemini_analyzer.py</code> and <code>diagram_consistency.py</code>). Distinct from <code>ANALYZE_CONCURRENCY</code> , which bounds whole jobs.	6	Local <code>.env</code> ; Azure env (plain)
AI_TIMEOUT	Per-request AI timeout, in seconds . Applied to the OpenAI client timeout and ($\times 1000$, as ms) to Gemini <code>http_options</code> .	90	Local <code>.env</code> ; Azure env (plain)
AI_RETRIES	Retry attempts on transient AI errors (HTTP 429/503/504) with linear backoff ($3 \times (\text{attempt}+1)$ seconds). Total attempts = <code>AI_RETRIES + 1</code> .	2	Local <code>.env</code> ; Azure env (plain)
DEV_USER_EMAIL	Local-dev identity fallback. Used by <code>resolve_identity()</code> only when no EasyAuth principal headers are present (i.e. no Entra sidecar). Ignored in production.	unset $\rightarrow$ <code>email</code> may be <code>None</code>	Local <code>.env</code> only
PLANSET_DATA_DIR	Root directory for the SQLite DB, uploads, runs, and generated artifacts (<code>db.py</code> sets <code>DATA_DIR</code>).	unset $\rightarrow$ <code>backend/data</code>	Local: unset; Azure env (plain), set to <code>/home/data</code> (Azure Files mount)
RULES_FILE	Overrides which rules YAML <code>rule_registry.py</code> loads. Absolute path, or relative to <code>backend/app/</code> . Missing file	unset $\rightarrow$ <code>rules.yaml</code> next to the module	Local <code>.env</code> ; Azure env (plain) if overriding

	logs a warning and falls back to the default. Lets you A/B rule sets without code changes.		
FRONTEND_DIST	Path to the built React SPA. When set and a valid directory, <code>main.py</code> mounts it at <code>/</code> so one container serves both API and UI (mounted last, after <code>/api</code> and <code>/artifacts</code>).	unset (local: Vite serves UI on <code>:5173</code>)	Set inside the Docker image / Azure; unset locally
ENABLE_TRAINING_RAG	Feature flag enabling the training-doc RAG path (<code>training_docs.py</code>). Truthy values: <code>1</code> , <code>true</code> , <code>yes</code> , <code>on</code> (any other value = off).	unset → disabled	Local <code>.env</code> ; Azure env (plain) if enabled

Notes on specific defaults

- **`OPENAI\_MODEL` default vs. production:** The code-level default is `gpt-4.1-mini` (and `.env.example` mirrors it). Production/sandbox override `OPENAI_MODEL` / `OPENAI_MODEL_DEEP` explicitly via Azure env vars to run the intended gpt-5.x models; the deep client comments note that these reasoning models ignore/reject `temperature`, which is why the code never sends one.
- **`AI\_PARALLELISM` in `.env.example`:** listed under "Tuning (optional)". It controls intra-analysis fan-out; keep it separate in your mental model from `ANALYZE_CONCURRENCY` (whole-job bound).

Authentication / EasyAuth configuration

The backend does **not** implement its own login. In production, the Azure Container Apps built-in authentication sidecar (Microsoft Entra, single-tenant, `castillope.com` org login) sits in front of the app and injects the signed-in identity as trusted request headers (it strips any client-supplied copies).

`backend/app/auth.py` reads them via the `current_user` FastAPI dependency.

Request headers consumed (set by the platform, not by you):

Header	Meaning
X-MS-CLIENT-PRINCIPAL-NAME	Principal name (typically the email/UPN); primary email source
X-MS-CLIENT-PRINCIPAL-ID	Stable principal/object id; primary <code>user_id</code> source

X-MS-CLIENT-PRINCIPAL

Base64-JSON claims blob, decoded for email (`preferred_username`, `upn`, `email`, `emails`, ...), display name (`name`), and object id (`oid`) claims when the direct headers are absent

Identity resolution order (`resolve_identity()` → {`email`, `user_id`, `name`}):

1. `email` = `X-MS-CLIENT-PRINCIPAL-NAME`, else first matching email claim from the decoded blob, else `DEV_USER_EMAIL`, else `None`.
2. `user_id` = `X-MS-CLIENT-PRINCIPAL-ID`, else an `oid`/object-identifier claim.
3. `name` = a `name` claim, else the resolved `email`.

The legacy free-text `engineer_name` form field is a last-resort display fallback only and is **not** authoritative.

Configuration ownership: The Entra app registration, allowed tenant/org, redirect URIs, and the "require authentication" toggle are configured on the Container App's Authentication (EasyAuth) blade by the DevOps/Automation Engineer — there is no application env var that turns auth on or off. Locally there is no sidecar, so set `DEV_USER_EMAIL` to simulate a signed-in engineer.

CORS: `main.py` configures `CORSMiddleware` with `allow_origins=["*"]` as a hard-coded value — there is no env var for it. Adjust in code if the policy needs tightening.

Local development configuration (backend/.env, gitignored)

Copy the template and edit:

```
cp backend/.env.example backend/.env
```

A minimal OpenAI local config:

```
# --- AI provider ---
AI_PROVIDER=openai
OPENAI_API_KEY=sk-...your-key...
OPENAI_MODEL=gpt-4.1-mini
# OPENAI_MODEL_DEEP=gpt-4.1          # optional heavier model for deep passes
# OPENAI_SEED=42                    # optional best-effort determinism

# --- Tuning (optional) ---
# AI_PARALLELISM=6
# AI_TIMEOUT=90
# AI_RETRIES=2

# --- Storage (leave unset locally -> backend/data) ---
```

```
# PLANSET_DATA_DIR=

# --- Local identity (no Entra sidecar locally) ---
DEV_USER_EMAIL=engineer@castillope.com
```

To use Gemini locally instead:

```
AI_PROVIDER=gemini
GEMINI_API_KEY=...your-key...
GEMINI_MODEL=gemini-2.5-flash
# GEMINI_MODEL_DEEP=          # optional; empty falls back to GEMINI_MODEL
```

Local runtime notes:

- Leave **PLANSET_DATA_DIR** unset; the SQLite DB, uploads, runs, and artifacts land under **backend/data**.
- Leave **FRONTEND_DIST** unset; run the API on **:8000** and Vite on **:5173**.
- Set **DEV_USER_EMAIL** so signed-in attribution and the Activity panel work without the Entra sidecar.

Azure configuration (Container Apps)

Split configuration by sensitivity:

- **Secrets** (Container App secrets, referenced by an env var): **OPENAI_API_KEY** — and **GEMINI_API_KEY** only if the Gemini provider is ever enabled.
- **Plain environment variables:** **AI_PROVIDER**, **OPENAI_MODEL**, **OPENAI_MODEL_DEEP**, **OPENAI_SEED** (when in use), **ANALYZE_CONCURRENCY**, **AI_PARALLELISM**, **AI_TIMEOUT**, **AI_RETRIES**, **RULES_FILE** (if overriding), **ENABLE_TRAINING_RAG** (if enabled), and the storage/serving vars.
- **Set by the image / platform, not per-environment:** **FRONTEND_DIST** (points at the built SPA inside the image so one container serves API + UI); **PLANSET_DATA_DIR=/home/data** (the mounted Azure Files share, so the SQLite DB and artifacts persist across revisions). **DEV_USER_EMAIL** is **not** set in Azure — identity comes from the EasyAuth headers.

Because the deployment runs a **single always-on replica**, the in-memory job registry/queue in **jobs.py** and the SQLite-on-Azure-Files store are safe. If the app is ever scaled to multiple replicas, **ANALYZE_CONCURRENCY**'s in-process queue and the SQLite store must move to a shared backing — do not simply raise the replica count.

Example: enable best-effort determinism in production

Setting **OPENAI_SEED** on the prod Container App (plain env var; triggers a new revision):

```
az containerapp update \
  --name castillo-qaqc-automation \
```

```
--resource-group castillo-qaqc-automation-rg \
--set-env-vars OPENAI_SEED=42
```

To revert to normal sampling, remove the variable (trailing -):

```
az containerapp update \
--name castillo-qaqc-automation \
--resource-group castillo-qaqc-automation-rg \
--remove-env-vars OPENAI_SEED
```

Apply the same commands against `castillo-qaqc-sandbox` (with its resource group) to configure the sandbox environment. Secrets such as `OPENAI_API_KEY` are managed with `az containerapp secret set` and referenced by an env var using `secretref:` — never passed inline on the command line.

Model selection: per-page (standard) vs. deep

Every public entry point in `gemini_client.py` (`analyze_page_image`, `analyze_multiple_images`, `analyze_text`, `analyze_document`) takes a `deep: bool` flag. That flag chooses the model:

- **Standard / per-page checks** (`deep=False`) → `OPENAI_MODEL` (or `GEMINI_MODEL`). Used for the high-volume, page-by-page vision passes.
- **Deep checks** (`deep=True`) → `OPENAI_MODEL_DEEP` (or `GEMINI_MODEL_DEEP`). Used for heavier reasoning: single-line-diagram validation, cross-sheet consistency, and supporting-doc/tech-spec cross-checks.

Selection is provider-symmetric:

```
def _pick_openai_model(deep): return _OPENAI_MODEL_DEEP if deep else _OPENAI_MODEL
def _pick_gemini_model(deep): return _GEMINI_MODEL_DEEP if deep else _GEMINI_MODEL
```

If the `*_DEEP` variable is empty, the deep path transparently reuses the standard model — so you can run a single model everywhere by setting only `OPENAI_MODEL` (or `GEMINI_MODEL`) and leaving the `_DEEP` var unset. To confirm what is live at runtime, `get_active_models()` returns `{provider, standard, deep}` for logging.

Related source files (absolute paths)

- `D:\code\QAQC\planset-qc-app\backend\app\gemini_client.py` — provider selection, model picking, `OPENAI_SEED`, `AI_TIMEOUT`, `AI_RETRIES`
- `D:\code\QAQC\planset-qc-app\backend\app\jobs.py` — `ANALYZE_CONCURRENCY`, bounded job queue

- D:\code\QAQC\planset-qc-app\backend\app\gemini_analyzer.py /
diagram_consistency.py — AI_PARALLELISM
- D:\code\QAQC\planset-qc-app\backend\app\auth.py — EasyAuth headers, DEV_USER_EMAIL
- D:\code\QAQC\planset-qc-app\backend\app\rule_registry.py — RULES_FILE
- D:\code\QAQC\planset-qc-app\backend\app\db.py — PLANSET_DATA_DIR
- D:\code\QAQC\planset-qc-app\backend\app\main.py — FRONTEND_DIST, CORS
- D:\code\QAQC\planset-qc-app\backend\app\training_docs.py — ENABLE_TRAINING_RAG
- D:\code\QAQC\planset-qc-app\backend\.env.example — local template